Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.:T001	Name:Soham Acharekar
Class:TYIT	Batch:1
Date of Assignment:14/7/26	Date/Time of Submission:14/7/26

Part A - Theory :

C# Delegate

What is a Delegate?
A **Delegate** is a special type in C# that stores the reference (address) of a method. It allows a method to be called indirectly through a delegate object. A delegate can call any method that has the **same return type and parameters** as the delegate declaration.
Why are Delegates Used?

- To call methods indirectly.
- To increase code flexibility.
- To implement callbacks.
- Used in event handling and GUI programming.

Real-Life Example
Think of a **TV Remote**.

- TV = Method
- Remote = Delegate

Instead of pressing buttons on the TV, we use the remote. Similarly, instead of calling a method directly, we use a delegate.

Syntax of Delegate
delegate returnType DelegateName(parameter_list);
Example
delegate void Operation(int a, int b);

Steps to Use a Delegate

Step 1: Declare the delegate

```
delegate void Operation(int a, int b);
```

Step 2: Create methods having the same signature

```
public void Add(int a, int b)
{
   Console.WriteLine(a + b);
}
```

Step 3: Create a delegate object

```
Operation obj;
```

Step 4: Assign a method to the delegate

```
obj = Add;
```

Step 5: Invoke the delegate

```
obj(10, 20);
```

or

```
obj.Invoke(10, 20);
```

Part B - Program

a) Create methods add(). multiply(), substract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

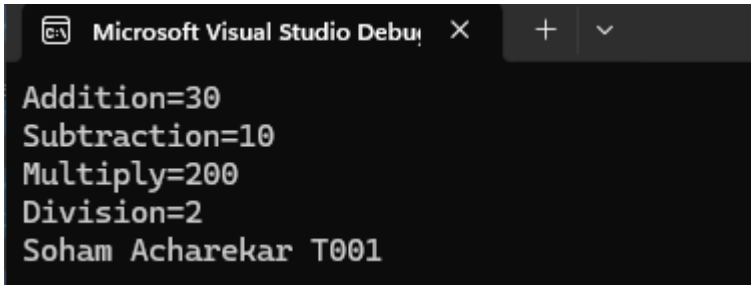
1. Start Declare a delegate named Operation with parameters (int a, int b).
2. Create a class Calculator containing methods:
   - Add(int a, int b) → performs addition.
   - Sub(int a, int b) → performs subtraction.
   - Multiply(int a, int b) → performs multiplication.
   - Division(int a, int b) → performs division.
3. Define Program class with Main() method.
4. Create an object of Calculator class.
5. Declare a delegate reference Operation obj.
6. Assign methods to delegate one by one:
   - obj = c.Add; → invoke with (20, 10).
   - obj = c.Sub; → invoke with (20, 10).
   - obj = c.Multiply; → invoke with (20, 10).
   - obj = c.Division; → invoke with (20, 10).
7. Display results of each operation.
8. End

Code:

```
using System;
delegate void Operation(int a,int b);
class Calculator
{
   public void Add(int a,int b)
   {
      Console.WriteLine("Addition="+(a+b));
   }
   public void Sub(int a,int b)
   {
      Console.WriteLine("Subtraction=" + (a-b));
   }
   public void Multiply(int a, int b)
   {
      Console.WriteLine("Multiply=" + (a*b));
   }
   public void Division(int a, int b)
   {
      Console.WriteLine("Division=" + (a/b));
   }
}
class Program
{
   static void Main()
   {
      Calculator c = new Calculator();
      Operation obj;
      obj = c.Add;
      obj(20, 10);
```

```
        obj = c.Sub;
        obj(20, 10);
        obj = c.Multiply;
        obj(20, 10);
        obj = c.Division;
        obj(20, 10);
        Console.WriteLine("Soham Acharekar T001");
    }
}
```

Output:



b) Write a program using multicast delegate.

Algorithm:

1. Start Declare a delegate named Message with no parameters and no return type.
2. Create a class Demo containing methods:
    - Welcome() → prints "Welcome Students".
    - Learn() → prints "Learn Delegates".
    - ThankYou() → prints "Thank You".
3. Define Program class with Main() method.
4. Create an object of Demo class.
5. Declare a delegate reference Message obj.
6. Assign methods to delegate:
    - obj = d.Welcome;
    - obj += d.Learn;
    - obj += d.ThankYou;

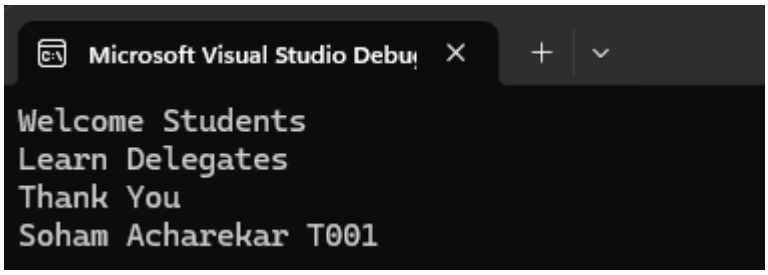
    (This combines multiple methods into one delegate → multicast delegate).
7. Invoke the delegate obj(); → executes all assigned methods in sequence.
8. End

Code:

```
using System;
delegate void Message();
class Demo
{
    public void Welcome()
    {
        Console.WriteLine("Welcome Students");
    }
    public void Learn()
    {
        Console.WriteLine("Learn Delegates");
    }
    public void ThankYou()
    {
        Console.WriteLine("Thank You");
    }
```

```
}
class Program
{
   static void Main()
   {
      Demo d = new Demo();
      Message obj;
      obj = d.Welcome;
      obj += d.Learn;
      obj += d.ThankYou;
      obj();
      Console.WriteLine("Soham Acharekar T001");
   }
}
```

Output:



c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

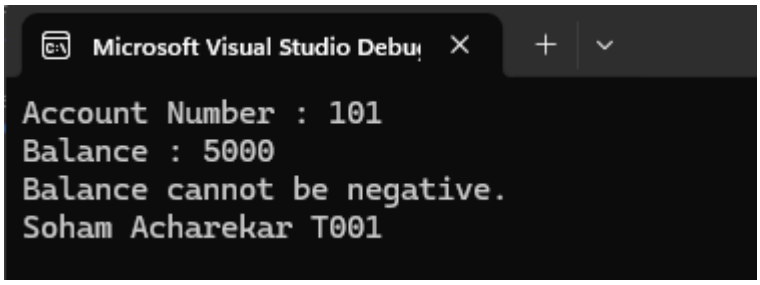
1. Start Define a class BankAccount with:
   - Property AccountNumber (public, auto-implemented).
   - Private field balance.
   - Property Balance with get and set:
   - get → returns current balance.
   - set → updates balance only if value ≥ 0, otherwise prints "Balance cannot be negative.".
2. Define Program class with Main() method.
3. Create an object b of BankAccount.
4. Assign values:
   - b.AccountNumber = 101;
   - b.Balance = 5000;
5. Display account details:
   - Print "Account Number : 101".
   - Print "Balance : 5000".
6. Try to set a negative balance:
   - b.Balance = -1000;
   - Output → "Balance cannot be negative.".
7. End

Code:

```
using System;
class BankAccount
{
   public int AccountNumber { get; set; }
   private double balance;
   public double Balance
```

```
    {
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
                balance = value;
            else
                Console.WriteLine("Balance cannot be negative.");
        }
    }
}
class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 101;
        b.Balance = 5000;
        Console.WriteLine("Account Number : " + b.AccountNumber);
        Console.WriteLine("Balance : " + b.Balance);
        b.Balance = -1000;
        Console.WriteLine("Soham Acharekar T001");
    }
}
```

Output:



Part C – Curiosity Question

1. A college wants to **calculate student performance**.
    a. Create methods to
    b. Calculate Total Marks
    c. Calculate Percentage
    d. Display Grade
    e. Display Pass/Fail

    Note: Call each method using a delegate.

ALGORITHM:

1. Start Declare a delegate named StudentDelegate with no parameters and no return type.
2. Create a class Student with:
    - Field total (int).
    - Field percentage (double).
    - Method CalculateTotal() → adds marks of three subjects (70, 80, 90) and prints total.
    - Method CalculatePercentage() → calculates average percentage and prints it.
    - Method DisplayGrade() → prints grade based on percentage:
        - ≥ 75 → Grade A
        - ≥ 60 → Grade B
        - ≥ 50 → Grade C
        - Else → Fail
    - Method DisplayResult() → prints Pass if percentage ≥ 50, else Fail.
3. Define Program class with Main() method.
4. Create an object s of Student.
5. Declare a delegate reference StudentDelegate obj.
6. Assign methods to delegate:
    - obj = s.CalculateTotal;
    - obj += s.CalculatePercentage;
    - obj += s.DisplayGrade;
    - obj += s.DisplayResult;

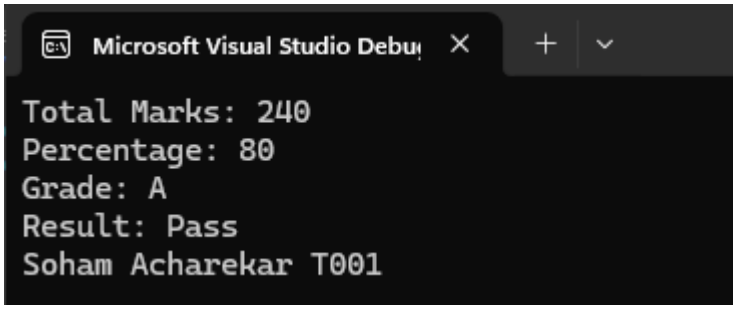
    (This makes obj a multicast delegate).
7. Invoke delegate obj(); → executes all assigned methods sequentially.
8. Print student details (Soham Acharekar T001).
9. End

CODE:

```
using System;
delegate void StudentDelegate();
class Student
{
   int total;
   double percentage;
```

```
    public void CalculateTotal()
    {
        int m1 = 70, m2 = 80, m3 = 90;
        total = m1 + m2 + m3;
        Console.WriteLine("Total Marks: " + total);
    }
    public void CalculatePercentage()
    {
        percentage = total / 3.0;
        Console.WriteLine("Percentage: " + percentage);
    }
    public void DisplayGrade()
    {
        if (percentage >= 75)
            Console.WriteLine("Grade: A");
        else if (percentage >= 60)
            Console.WriteLine("Grade: B");
        else if (percentage >= 50)
            Console.WriteLine("Grade: C");
        else
            Console.WriteLine("Grade: Fail");
    }
    public void DisplayResult()
    {
        if (percentage >= 50)
            Console.WriteLine("Result: Pass");
        else
            Console.WriteLine("Result: Fail");
    }
}
class Program
{
    static void Main()
    {
        Student s = new Student();
        StudentDelegate obj;
        obj = s.CalculateTotal;
        obj += s.CalculatePercentage;
        obj += s.DisplayGrade;
        obj += s.DisplayResult;
        obj(); // Calling all methods using delegate
        Console.WriteLine("Soham Acharekar T001");
    }
}
```

OUTPUT:



2. A college wants to automate the admission process.
   When the user clicks **Confirm Admission**, several operations must happen automatically. Design a delegate-based solution.

Students decide:

a. Verify Documents
b. Generate Roll Number
c. Allocate Division
d. Send SMS

Note: For above operation just print a statement.

ALGORITHM:

1. Start Declare a delegate named AdmissionDelegate with no parameters and no return type.
2. Create a class Admission with methods:
   - VerifyDocuments() → prints "Documents Verified".
   - GenerateRollNumber() → prints "Roll Number Generated".
   - AllocateDivision() → prints "Division Allocated".
   - SendSMS() → prints "SMS Sent to Student".
3. Define Program class with Main() method.
4. Create an object a of Admission.
5. Declare a delegate reference AdmissionDelegate obj.
6. Assign methods to delegate:
   - obj = a.VerifyDocuments;
   - obj += a.GenerateRollNumber;
   - obj += a.AllocateDivision;
   - obj += a.SendSMS;
     (This makes obj a multicast delegate).
7. Print message "Confirm Admission Clicked...".
8. Invoke delegate obj(); → executes all assigned methods sequentially.
9. Print student details (Soham Acharekar T001).
10. End

CODE:

```
using System;

delegate void AdmissionDelegate();

class Admission
{
   public void VerifyDocuments()
   {
      Console.WriteLine("Documents Verified");
   }

   public void GenerateRollNumber()
   {
      Console.WriteLine("Roll Number Generated");
   }

   public void AllocateDivision()
   {
      Console.WriteLine("Division Allocated");
   }

   public void SendSMS()
   {
      Console.WriteLine("SMS Sent to Student");
   }
```

```
}

class Program
{
   static void Main()
   {
      Admission a = new Admission();

      AdmissionDelegate obj;

      obj = a.VerifyDocuments;
      obj += a.GenerateRollNumber;
      obj += a.AllocateDivision;
      obj += a.SendSMS;

      Console.WriteLine("Confirm Admission Clicked...\n");

      obj(); // Calling all operations using delegate
      Console.WriteLine("Soham Acharekar T001");
   }
}
```

OUTPUT: